LocalStat: A Native macOS Environment for Local Execution of a SAS-Language Subset

Muhammad Hafiz bin Ismail

4 November 2026
Draft manuscript with baseline measurements

Abstract

LocalStat combines a native macOS editing environment with a local interpreter for a supported subset of the SAS language. The application uses Swift and AppKit for document editing, execution control, diagnostics, and result presentation. Its runtime is a pinned and locally extended version of OpenSAS, implemented in Zig. Local extensions include a bounded ordinary least-squares implementation of PROC REG and structured table output for selected procedures. This paper describes the system architecture, execution model, numerical implementation, and compatibility boundaries. The evaluation comprises 703 fresh-process command-line samples and 152 native runner and application samples on an Apple M5 Mac with 16 GB of memory. Each configuration has fifteen measured repetitions after a separately recorded first run and three warmups. Across seven one-million-row workload families, median command-line latency ranges from 0.877 to 1.678 seconds and median peak resident memory ranges from 1.43 to 1.96 GiB. Displaying 10,000 PRINT rows takes a median of 385 ms through the application, compared with 107 ms through its native runner. Separate stack samples identify repeated CSV conversion, field lookup, and allocation work. These are baseline observations; the study establishes neither optimization speedups nor general equivalence to SAS.

1 Introduction

LocalStat provides a macOS environment for editing and executing supported SAS programs locally. The application combines source editing, project navigation, execution controls, diagnostics, text output, formatted tables, and report export. Once built, the application runs without a developer toolchain or a hosted execution service.

The system uses OpenSAS as its interpreter foundation [1]. LocalStat adds a native application layer and selected runtime extensions. The implementation is independent of SAS Institute's runtime and supports a bounded language subset. Compatibility must therefore be described through specific supported behavior rather than inferred from the ability to execute a program successfully.

This paper documents three implementation areas: the integration of a native editor with a local interpreter, a bounded regression procedure, and a structured interface for presenting selected procedure results. It also reports a reproducible baseline evaluation to guide subsequent optimization.

2 System Architecture

2.1 Native application

The application is implemented in Swift using AppKit. Its interface includes document tabs, a project sidebar, an editor, and results views. Users can run an entire editor buffer, including unsaved changes, with the selected project directory serving as the runtime working directory.

The application retains the executed source snapshot with the run result. Diagnostic navigation is enabled only when an original-source position refers to an unchanged editor buffer. Positions belonging to macro-expanded text are not treated as direct coordinates in the original source.

Offline Help documents supported behavior and provides executable examples. Text export preserves runtime output and diagnostics. PDF report export combines the executed source, run metadata, results, and diagnostics. This report facility belongs to the application and is distinct from a SAS ODS destination.

2.2 Interpreter process

Each submission starts a fresh interpreter process. The application writes the source snapshot to a temporary file, launches the bundled executable, and drains standard output and standard error concurrently. Concurrent draining prevents a full output pipe from blocking completion.

Fresh processes reset WORK datasets and macro definitions between submissions. Files explicitly written by a program can persist. The process boundary separates run state; it does not provide a filesystem sandbox.

The application supports cancellation through a termination signal, followed by forced termination after two seconds if the process remains active. Runtime output is buffered until execution finishes, so cancellation can discard output that has not yet been emitted.

2.3 Runtime provenance and build

The interpreter is based on OpenSAS commit `927a2fa9f1401e365d975536bd704819ada1f1f4`. The repository retains the upstream source snapshot and records local modifications separately [2].

The current build targets Apple Silicon and macOS 14 or newer. The runtime is built with Zig 0.16.0 using `ReleaseSafe`; the Swift application uses a release build. Python supports repository tests and Help generation. It is not part of the application execution path.

3 Selected Implementation Details

3.1 Bounded ordinary least squares

LocalStat extends the runtime with a numeric subset of `PROC REG`. A supported model has one response, explicit numeric predictors, and an intercept:

$$y = X\beta + \varepsilon.$$

Rows missing any model variable are excluded. The remaining design must have positive residual degrees of freedom and sufficient numerical rank.

The solver uses column scaling and Householder QR factorization with column pivoting. It produces coefficient estimates, standard errors, analysis-of-variance quantities, and associated test statistics.

The implementation rejects numerically rank-deficient designs. It does not reproduce generalized-inverse handling of aliased coefficients. Unsupported model options produce explicit diagnostics. This procedure is a documented subset rather than a complete implementation of SAS PROC REG [3].

3.2 Structured table results

The runtime can write an optional, versioned JSON sidecar for selected results from `PROC PRINT`, `PROC FREQ`, and `PROC TABULATE`. The sidecar contains table titles, headers, rows, notes, and cell spans.

The application validates the version and table shape before displaying the results. Raw output and diagnostics remain available alongside formatted tables. The interface supports native table presentation without interpreting aligned text as a table schema.

4 Evaluation Questions

The evaluation addresses four questions:

1. How do interpreter runtime and peak memory change as input size increases?
2. Which runtime functions and execution stages account for the largest costs?
3. How much additional latency comes from table generation, result decoding, and application presentation?
4. Do targeted optimizations improve performance while preserving validated behavior?

The study does not attempt to establish that one implementation language is universally faster than another. Such a comparison would require equivalent implementations and controlled workloads. The first three questions are addressed through baseline measurements and profiles. The fourth remains open: no optimized implementation was evaluated.

5 Experimental Method

5.1 Workloads

A six-observation multiple-regression example from the repository, with an appended checksum, provides a small-program baseline. Synthetic workloads use 1,000, 10,000, 100,000, and 1,000,000 rows. Every generated CSV has five numeric columns: a descending row identifier, a group identifier, two predictors, and a response. The group identifier cycles through 100 levels. Predictors and noise cycle through a balanced factorial design with values -1 and 1, and the response is $y = 2 + 3x - 4z + \varepsilon$.

The seven scaling families perform a DATA-step transformation ($v = 3\,\mathrm{id} + 2$), ascending sort, SQL grouping, SQL join, two-predictor regression, CSV round trip, or frequency tabulation. The grouping workload produces 100 groups. The join uses a 100-row dimension with unique group keys and produces exactly as many rows as its input. The CSV round trip imports, exports, and reimports all five columns. PRINT adds output-heavy cases at 1,000 and 10,000 rows. Both FREQ and PRINT run with and without the structured table sidecar.

Input generation occurs outside the timed interval. Each scaling program includes CSV loading and a compact validation scan; these measurements are whole-program costs, not isolated operator timings. Checks verify row counts and exact checksums, sort order, group aggregates, and joined key associations. Regression checks require coefficients $(2, 3, -4)$ within absolute and relative tolerances of 10^{-8}, and residual sum of squares equal to the row count within relative tolerance 10^{-8}. Every exported CSV field and numeric listing row is checked after timing. Sidecar checks validate shape, PRINT cells, and FREQ group counts. Output and diagnostic hashes must remain identical across repetitions and sidecar variants.

5.2 Measurement boundaries

Command-line measurements run the executable directly. A high-resolution monotonic clock surrounds process launch and `wait4`; elapsed time includes launch and observation overhead, source loading, interpretation, and output completion. Per-child `wait4` resource counters provide CPU time and peak resident memory. This replaces the draft protocol's `/usr/bin/time -l`

invocation to retain sub-centisecond precision and avoid a separate timing process. Compilation, input generation, output parsing, and validation are excluded.

The native experiment compares the existing `LocalRuntimeRunner` directly against the actual `AppDelegate.runProgram` workflow in a release-mode XCTest host. Both execute the same frozen interpreter. The direct runner includes source snapshot creation, process handling, output capture, and table decoding. Application timing starts before dispatch and ends after result application, layout, and a synchronous window display pass. Completion observation polls at one-millisecond intervals. Window creation and document setup are excluded. The four cases are DATA at 10,000 rows, FREQ at 100,000 rows, and PRINT at 1,000 and 10,000 rows.

These are paired executions, not separately instrumented stages of one execution. Their difference includes scheduling and observation effects and cannot isolate decoding or rendering time. The display boundary does not measure physical screen presentation or the latency of a human keyboard action.

Separate one-second stack profiles were collected at nominal one-millisecond sampling intervals for each million-row family. Profiled runs are excluded from timing summaries. Their short, partially observed execution windows support hotspot identification rather than precise stage-time attribution.

5.3 Repetition and environment

Each configuration has one separately recorded first execution, three warmups, and fifteen measured repetitions in fresh interpreter processes. The primary summaries are medians and interquartile ranges (IQRs), using linearly interpolated inclusive quartiles. The first runs and warmups are retained in the raw artifact but excluded from summary tables. No cache flush was performed, and first execution is not claimed to be a cold-cache measurement. Paired variant order alternates by iteration.

The host was a ten-core Apple M5 MacBook Pro (Mac17,2) with 16 GB of memory, running macOS 26.4.1 (25E253), Swift 6.3.3, and Zig 0.16.0. The Mac was connected to AC power with low-power mode disabled. The runtime used the normal ARM64 ReleaseSafe build; the native test host used Swift release compilation. App windows used the light appearance, a 1100×780 frame, default fonts, and unit table zoom. Timing studies and profiling ran sequentially.

The artifact records executable hashes, source hashes, working-tree state, compiler versions, power settings, and load averages. Native app sources were frozen before compilation to prevent concurrent workspace edits from changing that experiment. The machine was not otherwise isolated from operating-system or user activity. Command-line runs used the automation filesystem sandbox; native app runs required normal AppKit access. The native runner and application comparison used the same host process and access conditions.

5.4 Correctness and comparisons

A sample is accepted only when execution succeeds and all applicable checks pass. A sixty-second command-line watchdog kills the process group on timeout; failed and timed-out runs are recorded rather than included as successful observations. Native output hashes are checked against validated command-line output. The reporting tool recomputes summaries from raw records and rejects incomplete, duplicated, or invalid repetitions.

For paired sidecar and native comparisons, the reported difference is the median of iteration-matched differences, not the difference of marginal medians. No significance test or confidence interval is asserted. An IQR crossing zero is treated as insufficient evidence of a consistent directional effect in that comparison. No optimized build was tested.

6 Results

6.1 Interpreter latency and memory

All 703 command-line samples passed validation, with no timeouts. There are 37 configurations and 555 measured samples; the other 148 are first executions or warmups. Table 1 reports the three larger sizes. The six-row example has a median latency of 2.7 ms (IQR 0.7 ms), including launch and observation costs.

At one million rows, median latency ranges from 877.2 ms for SORT to 1677.8 ms for the CSV round trip. Increasing input from 100,000 to one million rows increases median latency by factors of approximately 9.5 to 11.8 across these families. These observations describe this input range; they do not establish asymptotic complexity.

Median peak resident memory at one million rows ranges from 1464.5 MiB for FREQ to 2005.5 MiB for the SQL join. The shared CSV load, dataset representation, and validation scans are part of each workload. The table therefore cannot attribute either time or memory solely to a named procedure. The CSV round trip has a 273.0 ms latency IQR at one million rows, illustrating variation that a single execution would conceal.

6.2 Structured table output

Table 2 compares the sidecar variants. The paired median increase for PRINT is 0.6 ms at 1,000 rows and 4.3 ms at 10,000 rows. FREQ produces only 100 category rows regardless of input size. For FREQ at 10,000 through one million input rows, the paired-difference IQR crosses zero, so these observations do not establish a consistent directional sidecar cost.

6.3 Native application latency

All 152 native samples passed checks, including 120 measured samples across eight configurations. Table 3 compares the direct native runner and complete app workflow. Native runner costs include Foundation process management and table decoding and are distinct from the command-line launch path.

For the DATA and FREQ cases, paired median application-minus-runner differences are 7.7 and 8.0 ms. For PRINT, the difference grows from 32.9 ms at 1,000 rows to 276.4 ms at 10,000 rows. The latter case has a median application latency of 385.1 ms and an IQR of 75.7 ms. This identifies result-heavy application behavior as a separate optimization target, but does not isolate the cost of formatting, pagination, layout, drawing, or scheduling.

Workload	10,000	100,000	1,000,000	MiB
DATA transform	12.0 (0.7)	107.6 (6.8)	1035.0 (52.5)	1832.7
SORT	10.7 (4.6)	85.7 (4.0)	877.2 (29.1)	1533.0
SQL grouping	10.3 (0.9)	93.2 (5.5)	883.4 (21.9)	1614.7
SQL join	15.5 (0.7)	137.8 (3.3)	1346.6 (62.4)	2005.5
REG (two predictors)	10.8 (0.7)	94.1 (4.3)	1045.0 (100.3)	1625.4
CSV round trip	16.7 (2.2)	152.6 (10.3)	1677.8 (273.0)	1994.2
FREQ	10.0 (0.6)	80.3 (1.8)	949.2 (109.6)	1464.5

Table 1: Command-line median latency in milliseconds, with IQR width in parentheses; fifteen measured repetitions per cell. The last column is median peak resident memory in MiB at one million rows. Sidecar output is disabled.

Workload	Plain	Sidecar	Paired Δ
FREQ (1,000)	2.8 (0.1)	3.0 (0.1)	0.2 (0.2)
FREQ (10,000)	10.0 (0.6)	10.0 (0.9)	-0.2 (1.0)
FREQ (100,000)	80.3 (1.8)	80.7 (1.6)	0.0 (1.0)
FREQ (1,000,000)	949.2 (109.6)	932.6 (100.7)	14.1 (68.9)
PRINT (1,000)	3.6 (0.4)	4.2 (0.3)	0.6 (0.3)
PRINT (10,000)	16.1 (3.0)	20.7 (0.9)	4.3 (2.0)

Table 2: CLI table-output comparison in milliseconds. Values are medians (IQR widths). The paired difference is sidecar minus plain execution, matched by iteration. It can differ in sign from the difference of marginal medians.

Workload	Native runner	Application	Paired Δ
DATA transform (10,000)	65.9 (2.2)	74.1 (2.3)	7.7 (3.6)
FREQ (100,000)	134.8 (3.3)	143.8 (6.9)	8.0 (5.7)
PRINT (1,000)	71.5 (1.5)	104.9 (2.7)	32.9 (5.3)
PRINT (10,000)	107.4 (2.2)	385.1 (75.7)	276.4 (76.3)

Table 3: Native runner and application latency in milliseconds: medians (IQR widths), fifteen measured repetitions per cell. The paired difference is application minus runner. App timing ends after the first synchronous display pass, not physical screen presentation.

6.4 Profiles and optimization priorities

All seven profiled million-row runs completed successfully. The samples repeatedly show `io.readDelimited`, numeric conversion, PDV field lookup, hash lookup, and memory allocation or initialization. The DATA profile also shows row consumption, expression evaluation, and dataset output. These observations support investigating CSV conversion, field access, and data representation before considering a language migration. They do not quantify an achievable speedup or establish a precise breakdown of total execution time.

7 Verification and Reproducibility

Repository documentation remains pinned to base source revision `7f6dec6` (full identifier: `7f6dec6537cc9e9ca9f9c37961b8b6811c5b7aff`). The evaluated working tree also contained uncommitted app changes. The artifact preserves exact native sources, per-file hashes, runtime executable hashes, workload source, input hashes, individual samples, and profiles rather than claiming a clean evaluated release commit.

The fresh project verification log records 969 runtime unit tests, 1,924 corpus programs, and 11 end-to-end fixtures passing. The Swift suite reports 48 tests with one opt-in benchmark test skipped and no failures; the benchmark test was subsequently enabled for the native study. The packaged-runtime checks, harness checks, Help checks, and repository text check also passed. Separate evidence-auditor tests check incomplete records, output mismatches, and iteration pairing. Earlier verification records remain historical evidence [4].

The measurement artifact is stored under `docs/paper/results/2026-11-04`. The harness generates inputs and runs the CLI study, an opt-in native test measures the app, and a reporting script audits raw evidence and generates the three tables in this paper. Instructions describe rebuilding and repeating the experiments. These checks establish reproducibility of the stated workflow, not correctness for every SAS program.

8 Limitations

LocalStat supports a subset of the SAS language and procedure options. Passing repository tests does not establish general SAS compatibility. Existing numerical checks include analytical and published-example comparisons; this manuscript reports no new comparison against an installed SAS engine.

The application uses a fresh process for every submission and does not provide a persistent interactive runtime session. Buffered output and in-memory result retention also affect cancellation behavior and large-output workloads.

The alpha application is not sandboxed. Programs can access files permitted to the application and can observe inherited environment values through supported functionality. Local execution therefore does not, by itself, establish security or suitability for regulated work.

Performance observations from one Apple Silicon machine have limited generality. Synthetic numeric inputs, 100-level groups, a bounded many-to-one join, and two regression predictors cannot represent every user program. Warm file caches, ordered workload execution, operating-system activity, and the automation access conditions can affect results. The native measurements use a release-mode XCTest host rather than an independently launched packaged app. One-second profiles are incomplete samples of execution, and no persistent-session, alternative-language, SAS engine, or optimized-build comparison was performed.

9 Conclusion

LocalStat integrates a native macOS editing environment with a locally extended OpenSAS interpreter. Its current design provides fresh-process execution, source-aware diagnostics, bounded ordinary least squares, and structured presentation for selected tables.

The baseline study validates every recorded workload result and separates interpreter measurements from native application measurements. At one million rows, the evaluated workload families have median CLI latencies below 1.7 seconds but peak resident memory between 1.4 and 2.0 GiB. Large PRINT results add a substantial application cost beyond the native runner. The next experiments should test targeted changes to conversion, field access, data representation, and result presentation. The current results provide a baseline for those comparisons; they do not demonstrate an optimization or a language advantage.

References

[1] OpenSAS contributors. *OpenSAS source code*. Pinned commit: `927a2fa9f1401e365d975536bd704819ada1f1f4`. OpenSAS source snapshot.

[2] LocalStat project. *Runtime provenance and local changes*. `docs/RUNTIME_CHANGES.md`. Source revision `7f6dec6`.

[3] LocalStat project. *LocalStat source-build compatibility*. `docs/COMPATIBILITY.md`. Source revision `7f6dec6`.

[4] LocalStat project. *Verification record*. `docs/VERIFICATION.md`. Source revision `7f6dec6`; historical records dated 29 October and 1 November 2026.